MSE | MASTER OF SCIENCE IN ENGINEERING

Hes·so

Development of a Genetic Algorithm for Fabric Waste Minimization

Master of Science HES-SO in Engineering

Orientation Microengineering

Lausanne, January 9, 2026

Océane Patiny

Supervisor

Prof. Éric Taillard

Declaration of authenticity

I hereby affirm that this assignment is my own written work and that I have used no other sources or aids other than those indicated. All passages that have been quoted from publications or paraphrased from these sources are indicated as such. Additionally, any assistance from artificial intelligence tools has been appropriately acknowledged and documented.

Date: January 9, 2026
Name: Océane Patiny

Use of AI

In the scope of this project, the free version of the AI tools Perplexity[1] and Consensus[2] were used in order to find relevant references for the state of the art. In the scope of the report, Claude Sonnet v. 4.5 was used to generate the outline of the abstract, suggest suitable word translations from french to english, and detect spelling and grammar errors. On another hand, the GitHub Copilot Pro[3][4] extension was used inside of the Visual Studio Code IDE as a help for the programming part of the project. It provided inline code suggestions as well as a chatbot.

[1] https://www.perplexity.ai/
[2] https://consensus.app/
[3] https://github.com/features/copilot
[4] The Pro version is accessible for free as a student.

Abstract

This project develops a genetic algorithm (GA) to optimize the placement of sewing pattern pieces on fabric, aiming to minimize the remaining waste. The implementation uses a raster model, which discretizes both the fabric and pattern pieces into pixel-based images. Pattern pieces are extracted from SVG files, with text annotations automatically removed. Each GA individual (gene) encodes the row offset, column offset, and cardinal orientation of each pattern piece. The fitness function implements a weighted sum of five normalized criteria: piece overlap, fabric length used, average row and column of the origin of the piece, and packing efficiency. A custom mutation algorithm iteratively translates pieces to improve fitness scores, while one-point crossover maintains population diversity. The population is divided into elite individuals (highest fitness) and diverse individuals (maximum distance to elite). Testing on geometric shapes and actual sewing patterns demonstrates successful implementation, with packing efficiencies ranging from 40-72%. The algorithm handles pieces with holes and enables recursive nesting. While current results (average packing of ~60%) fall short of commercial software (~80%) and human experts (~85%), the modular architecture provides a flexible tool for testing alternative fitness functions, crossover operators, and mutation strategies. All the code is available publicly on https://github.com/opatiny/fawami.

Contents

1 Introduction **5**

2 State of the Art **6**

3 The Genetic Algorithm **8**

4 Implementation **16**

5 Debug Strategies **23**

6 Results **26**

7 Discussion **35**

8 Perspectives **35**

9 Conclusions **37**

Bibliography **39**

1 Introduction

With the rise of fast fashion and the modern clothing industry, our relationship toward textile products has evolved dramatically. From invaluable goods, worn for decades, mended and repaired, which would sometimes even be passed down generations, garments have become short-term consumables. Insidiously, we are pushed toward shorter and shorter cycles of "buy, wear, throw away", which has catastrophic social and environmental consequences [1].
One approach to reduce the impact of the textile industry is to reduce the fabric waste generated during cutting stage, which was estimated to approximately 15 % in 2021 [2]. This can be done in various ways, for instance by changing the pattern designs to have as many overlapping edges as possible once the pattern pieces are placed on fabric. This is the principle behind the zero-waste philosophy [3]. Another approach is to find an optimal placement of pattern pieces on the desired material in order to reduce the amount of waste. This task would traditionally be done by a specialist, taking into account the various constraints, such as the pieces orientation. The process can however also be automated, by developing appropriate optimization algorithms. The issue of fitting as many 2D shapes as possible on a given surface is known as nesting, or 2D packing. Parts nesting is a common issue, that arises in fields such as laser cutting, CNC milling or shoe-making [4]. Hence, improvement to nesting algorithms can have a meaningful impact way beyond the textile industry. In all sectors, waste minimization corresponds to a smaller environmental impact, and an economic gain.
The nesting problem was shown to be NP-complete, which implies that an algorithm that systematically returns the optimal solution in a polynomial time does not exist (unless P = NP) [5]. Therefore, *heuristics* have to be used, which are algorithms that return a "good solution", without guaranty that it is the best one. Even so, finding a solution to this sort of problem requires a large computational power. Therefore, research in the field of nesting greatly intensified at the beginning of the 1990s with the rise of more powerful and affordable computers, and has been deeply investigated since then [6].
One type of heuristic which has already been applied to the specific issue of pattern pieces placement is genetic algorithms, which are based on the combination, mutation and selection of pieces arrangements in order to converge toward a good solution. In this project, we are precisely focusing on that topic. The aim is to implement a genetic algorithm from scratch, which takes an array of pattern pieces and the width and height of the fabric as inputs, and returns the optimal locations and cardinal orientations for each piece in order to minimize the waste. The goal is to understand how a complex problem like pattern pieces layout can be abstracted in order to input it to a genetic algorithm. The chosen approach is to discretize both the fabric and the pattern pieces into images composed of pixels (this is referred to as the raster model). The pieces can then be placed at desired locations on the fabric image, and the pieces overlap can be computed by counting the number of overlapping pixels. In order to make the program as flexible as possible, the following constraints were specified:

- Input file format for the pattern pieces: The pattern is a `.svg` file containing all of the pieces to place on the fabric
- The program has to handle cardinal orientations of the pieces.
- Pieces can contain holes. Smaller parts can be placed within the holes.

2 State of the Art

In this section, we aim to identify a few keypoints from the literature which are particularly relevant to our specific task. Key principles for nesting problems are presented, as well as some existing work using the raster model (discretizing the pattern pieces and fabric).

2.1 Legal and relaxed placement methods

We can identify two different approaches to place the pieces on the fabric: those that allow piece overlap during the solution process, and those that don't. These are respectively referred to as the "relaxed" and "legal" placement methods. Both of these approaches have their strengths and weaknesses, neatly summarized by Nielsen and Odgaard [6]:

> "[T]he relaxed placement methods has to handle a larger search space than the legal placement methods, but it is also clear that the search space of the relaxed methods do not exclude any optimal solutions. This is not true for most of the legal placement methods and it is also one of their weaknesses. On the other hand they can often produce a good legal solution very quickly."

Often-times, legal placement methods use the No-Fit-Polygon algorithm [6]. This algorithm allows to identify all the positions for a polygon so that it touches a previously placed polygon, but does not to intersect it. Using this approach leads to a compact solution, since all pieces placed on the fabric are touching. For the algorithms using a legal placement method, the placement problem can be transformed into an ordering problem. This means that instead of trying to find optimal an placement of the parts, the algorithm has to return an optimal sequence in which to introduce the pieces on the fabric. This leads to a smaller search space, and therefore a faster algorithm [7].

2.2 Pieces representation

They are two main ways of representing the pattern pieces: using polygons or a raster model. Using a polygonal approach results in an exact computation of the intersection. What is more, the intersection computation is independent of the pieces dimensions. This is not the case with the raster model, one of whose bigger flaws is that the overlap computation depends on the resolution (or granulometry) of the parts. One possibility to improve the performance is to increase the granulometry as the algorithm runs, which allows to have a first rapid approximation, and then refine it in the final iterations [8]. Another issue with the raster model is handling rotations of the pieces which are not a multiple of 90°. Indeed, since it will cause further approximations (which pixels belong to the piece or not?) [6]. On the other hand, the raster model has tow main advantages:

- The pieces can have any shape, not only polygonal. This is especially interesting for textile pattern pieces, which can have multiple curves and be concave.
- The overlap calculation has the same complexity wether the pieces have holes or not.

2.3 Fitness function

The fitness function returns a numeric score for a given individual, which expresses its quality. Overall, there does not appear to be a consensus on the best way to express the fitness in the case of pieces nesting. Here are a few of the solutions that were proposed in previously published work:

- Minimize the sum of the distances between parts, the sum of distances of the parts to the origin of the fabric, and the pieces overlap (weighted sum) [8]
- Minimize the length of fabric used and amount of residual scrap [7]
- Minimize the length of fabric used [9]
- Maximize the area of the largest scrap of fabric [10]

2.4 Mutation

Mutation consists in taking an already existing solution and applying small changes to it. Typically, in the case of pattern piece placement, the mutation of a solution is a combination of the three following operations [8, 10]:

- Translation of some of the pattern pieces
- Swapping the positions of two pieces
- Changing the orientation of some pieces

The pieces to be swapped or moved are selected at random.

2.5 Packing

The packing is the surface of the pattern pieces divided by their bounding rectangle. It is generally expressed as a percentage. Packing is considered as a measurement of the quality of a solution, and it is a value that is often provided in existing literature. Overall, it appears that the best results that are obtained in scientific publications or with specialized commercial software range from 70 and 80 % [6, 8, 9]. On the other hand, human experts reach a packing of around 85 % [9].

3 The Genetic Algorithm

3.1 What is a genetic algorithm?

The idea behind genetic algorithms (GAs) is based on Darwin's theory of evolution: nature selects the fittest individuals, hence passing "good genes" from one generation to another. In the case of a GA, the first step is to create an initial population of solutions to the problem, as well as to define a fitness function, which attributes a score to each solution. Then, a new generation of solutions is created, by modifying the individuals from the initial population, and the best solutions are selected. The process is then repeated, which should converge toward a better and better solution. To summarize, the generic structure of a GA is as follows:

- **Create an initial generation of individuals.** This will generally imply using a random generator to create solutions that are as diverse as possible (covering as much of the solution space as possible).
- Repeat until end condition is met:
  - **Attribute a score** to each individual using the predefined fitness function.
  - **Select the N best individuals.**
  - **Create new individuals** from the previous generation. Keep the previous best solutions in the new generation. The new individuals are generated by applying the two following operators:
    1. **Crossover** (see Figure 1): Combine two individuals to create a new one. For instance, take the beginning of one gene, and complete it with the end of another one (1-point crossover).
    2. **Mutation** (see Figure 2): Apply small modifications to a gene.
- Stop when the end condition is met. This could for instance be a fixed number of iterations of the algorithm, or a limit on the convergence of the fitness function.

The whole challenge when implementing a genetic algorithm is first to find an appropriate representation of the individuals. Then, the fitness function has to be defined. Finally, the crossover and mutation functions have to be implemented. There is an infinite amount of possibilities for these three functions, and the goal is to find some which will lead to the fastest possible convergence of the algorithm toward a good solution.

Oftentimes, these different functions have top-level parameters, which can be varied (for instance weights inside of the fitness function). They are parameters whose value cannot be deduced from the data that is being processed, and are commonly reffered to as *hyperparameters* [11]. These hyperparameters have a drastic impact on the results of the algorithm, and finding the optimal values for them is a complex problem on its own. The basic approach is to pursue a tedious process of trial and error until an acceptable compromise is obtained. However, some tools exist, which allow to provide the whole definition of the genetic algorithm, as well as a list of all the hyperparameters, and which will run an array of tests in order to find the best values for the parameters automatically. This is referred to as *hyperparameter optimization*.

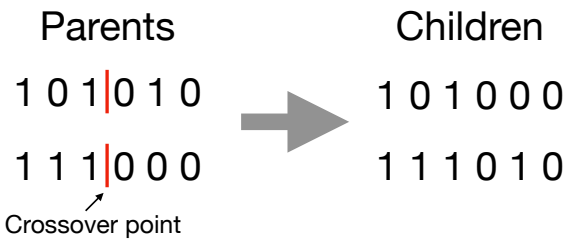


Figure 1: 1 point crossover of two 6 bits binary genes. The algorithm takes two parent genes and returns two children.

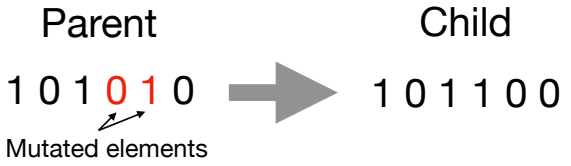


Figure 2: Random mutation of a 6 bits binary gene: the algorithm takes one parent gene and randomly modifies some of the elements of the gene, to generate an offspring slightly different from the parent.

3.2 Representation of the fabric and pieces

In the scope of this project, only rectangular fabric shapes are handled. Both the fabric and the pattern pieces to cut out are abstracted by matrices composed of discrete pixels. In the case of the pattern pieces, a corresponding binary mask is created, which encodes which pixels are occupied by the mask (value of 1), and which aren't (value of 0). This representation allows us to manipulate the pattern pieces as images, and therefore implement graphical debug tools (see section 5). What is more, the overlap between pieces becomes a simple sum of pixels (see subsection 4.5).

3.3 Defining the gene

First and foremost, the variables that vary from one gene (or individual) to another have to be identified. In our case, the goal is to optimize the origin and the orientation of an array of pattern pieces. For each individual of the genetic algorithm, the pattern pieces are the same. Therefore, for each piece, we have only three parameters that can vary:

- The **row offset** of the piece
- The **column offset** of the piece
- The **orientation** of the piece relative to the original orientation in the pattern.

Both offsets are integers (numbers of pixels), which express the position of the origin of the piece relative to the top-left corner of the fabric. In order to facilitate the handling of rotations, we have decided to define the origin of a piece in its center, i.e. at the center of its bounding rectangle (BR), as shown in Figure 3. This ensures that the origin of the piece's bounding rectangle remains the same even if the piece is rotated. Therefore, if a piece's orientation is mutated, the new piece is approximately at the same position as

the old one, which would not be the case with an origin in the top-left corner. What is more, the orientation is defined positive in the anti-clockwise direction, and is limited to the four cardinal orientations: 0, 90, 180 or 270°.

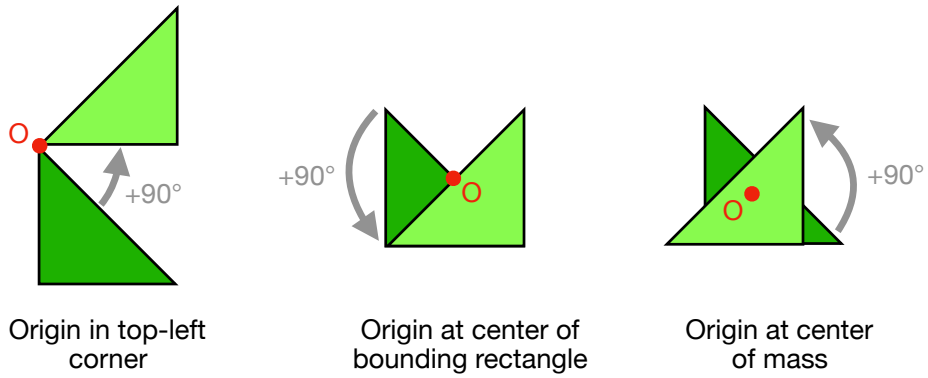


Figure 3: Various possibilities to define the origin of a pattern piece, and corresponding rotations. We implemented option 2: origin at the center of the BR.

We define the following notation for the location of a pattern piece:

$$(c, r, \alpha) \tag{1}$$

Where c is the column index, r is the row index, and α is the orientation of the piece. The column index can be seen as an X coordinate, and the row index as a Y coordinate, except that the axis points toward the bottom. What is more, another issue arises from the fact that the pattern pieces are discretised: sometimes the width or height has an even number of pixels, and there is therefore no exact center. In those cases, we consider the pixel which is the closest to the top-left corner of the piece as the center, as shown in Figure 4.

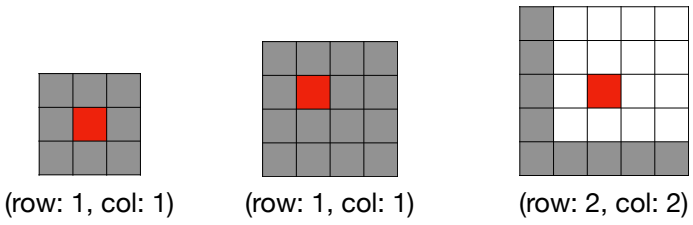


Figure 4: Example pattern pieces with their bounding rectangles. Pixels occupied by the piece are in gray. The corresponding center of each piece is in red.

Figure 5 shows a simple example in which two pattern pieces are placed on a square piece of fabric of 5x5 pixels. Piece 2 has an orientation of 90 degrees, and was rotated around its center.

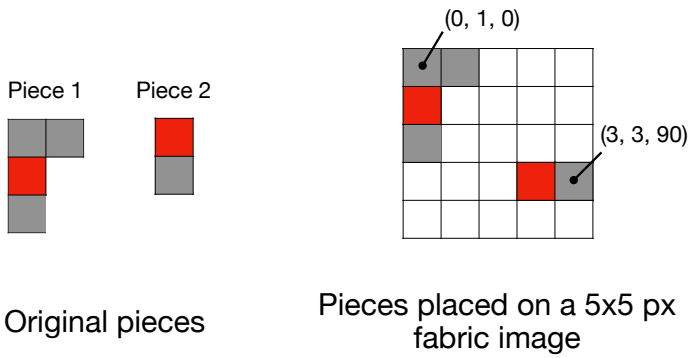


Figure 5: Example pattern pieces with their bounding rectangles. Pixels occupied by the piece are in gray, and the pieces centers are in red.

	Piece 1	Piece 2
Column	0	3
Row	1	3
Orientation	0	90

Table 1: Data corresponding to the gene illustrated in Figure 5

3.4 Fitness function

In order to create a fitness function adapted to the problem at stake, we must first specify what makes a solution "good". The following criteria were identified:

- All pieces must fit entirely within the fabric.
- The number of pixels o of the pieces which are overlapping must be minimal.
- The length L of fabric used must be minimal.
- Similar to the previous criterion: the average column c_{av} of the piece centers should be minimal. This pushes all the pieces toward the left, instead of only the right-most piece (if minimizing used length).
- In order to have the largest possible rectangular waste, the average row r_{av} can be minimized as well. This pushes the pieces toward the top of the fabric.
- The packing p of the solution should as high as possible. The packing is defined as the sum of the areas of the pieces divided by the area of the bounding rectangle of all the pieces.

In order to compute a single scalar from all these variables, we use a weighted sum[1]:

$$f = w_o \cdot o + w_L \cdot L + w_c \cdot c_{av} + w_r \cdot r_{av} + w_p \cdot (1 - p) \tag{2}$$

However, some of these criteria are interdependent: for instance a smaller length of fabric used will generally imply a smaller average column. Therefore, an appropriate combination of weights must be used inside of the fitness function.
Expressed as it is in equation 2, the fitness f is a value that must be minimized: the

[1]Corresponding implementation of the fitness function:
https://github.com/opatiny/fawami/blob/main/src/textileGA/getFitness.ts

smaller the fitness, the better the individual. What is more, the values of all of the weights are hyperparameters, which values will impact on the global result of the GA. As it is, a given set of weights might work well for a small image, but not for a large one (or conversely). Indeed, the values of the variables do not all change at the same rate depending on the image's dimensions. For instance, consider an image of $n \cdot n$ pixels. Then, the used length is proportional to n, whereas the overlap would be proportional to n^2. In order to eliminate this effect, the variables are normalized to have the same order of magnitude. Here is how we chose to normalize each of the variables:

- **Overlap:** The number of overlapping pixels is divided by the number of pixels in the fabric. This could result in a value larger than one.
- **Length of fabric used:** Divide the length of fabric used by the total length of fabric.
- **Average column of pieces:** The average column is divided by the fabric length.
- **Average row of pieces:** The average row is divided by the fabric width.
- **Packing:** It is already a value between 0 and 1 (could be larger than 1 if there is a lot of overlap).

In order to ensure that the pieces are within the fabric, their positions are clamped based on the fabric's dimension. Therefore, the pieces are always fully within the fabric, for all solutions. Another issue that arises, is that a solution is only acceptable if the overlap is zero. Hence, a solution which does not meet this criterion has to be rejected.

3.5 Creation of the initial population

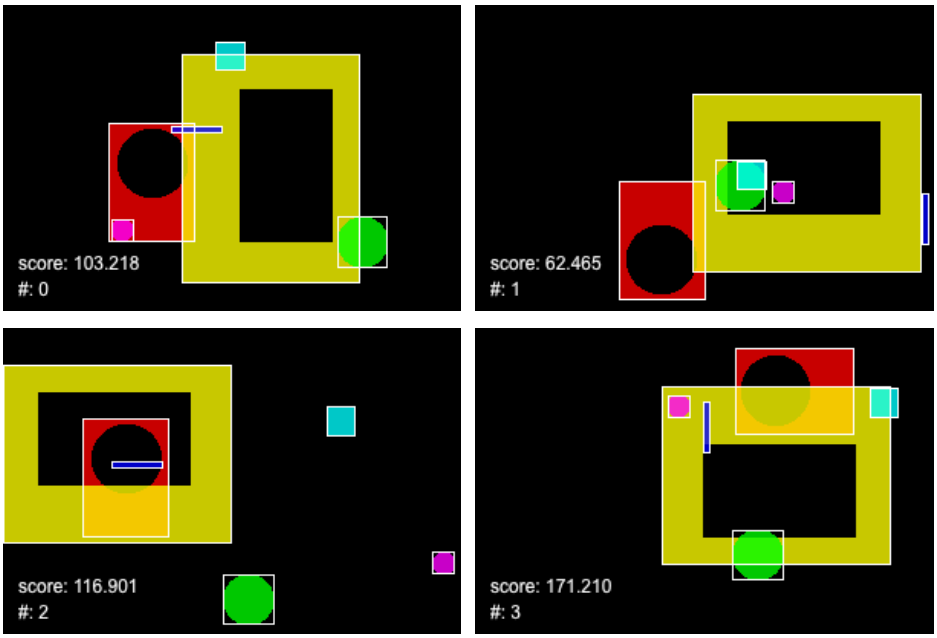


Figure 6: Four random initial individuals, which can be used as the first generation of the GA. The cardinal orientation of the pieces is attributed randomly. These orientations are not modified later on, since our mutation algorithm only translates the pieces.

The user can input an initial population to the algorithm if desired, however if none is provided, it will be computed automatically. The default algorithm attributes random positions and cardinal rotations to the parts, but ensures all parts fit within the fabric by clamping the positions.[1] An example of some random individuals is shown in Figure 6.

3.6 Crossover

A simple 1-point crossover was implemented: a random crossover point is chosen, then the tails of each genes are swapped[2]. A minimal size of the crossover can be specified by the user. For example, if a minimal crossover fraction of 0.2 is required for a gene with five pieces, there will be at least one piece swapped. The crossover is illustrated in Figure 7. Overall in the genetic algorithm, the crossover is used to bring **diversity** to the new solutions.

Parent 1

Piece	Row	Column	Orientation
0	1	1	0
1	3	4	90
2	2	1	180
3	5	2	90

Parent 2

Piece	Row	Column	Orientation
0	0	3	0
1	4	2	0
2	1	5	0
3	0	0	270

Child 1

Piece	Row	Column	Orientation
0	1	1	0
1	3	4	90
2	1	5	0
3	0	0	270

Child 2

Piece	Row	Column	Orientation
0	0	3	0
1	4	2	0
2	2	1	180
3	5	2	90

Figure 7: Example of the crossover of two four-pieces genes, and a crossover point of 2.

3.7 Mutation

An algorithm of "smart" mutation was implemented[3], which iteratively computes a slightly modified individual and then verifies whether the modification yielded to an improvement, until the individual cannot be improved anymore. The solutions are only modified by translating one piece at a time, which means that the orientations of the original gene are preserved. The user can define a "translation amplitude" in pixels, which indicates how much a piece is moved at every little modification. The idea behind the algorithm is the following:

1. Pick the piece to translate.
2. Pick the direction in which to move. The order of the directions is up, left, down, right.

[1]The function to generate the random genes:
https://github.com/opatiny/fawami/blob/main/src/textileGA/getRandomGenes.ts
[2]Crossover function: https://github.com/opatiny/fawami/blob/main/src/textileGA/crossover1Point.ts
[3]See the implementation of smart mutation:
https://github.com/opatiny/fawami/blob/main/src/textileGA/smartMutate.ts

3. Move the piece in the chosen direction by the amplitude given by the user.
4. Compile the modified gene's fitness score. If the score was improved, go to step 3, else, go to step 2.
5. Once the four directions were processed for the current piece, go to step 1.
6. Once all the pieces were processed, we consider that one iteration was completed. If no improvement of the score was made over a whole iteration, exit early, otherwise, repeat until maximum number of iterations is reached.

This algorithm allows to highly improve the score of a given solution. Therefore, it is the part of the GA which brings **quality** to the results. An example of a gene before and after smart mutation is illustrated in Figure 8.

Remark 1 *The output of the smart mutation algorithm depends on the order in which the pieces are processed.*

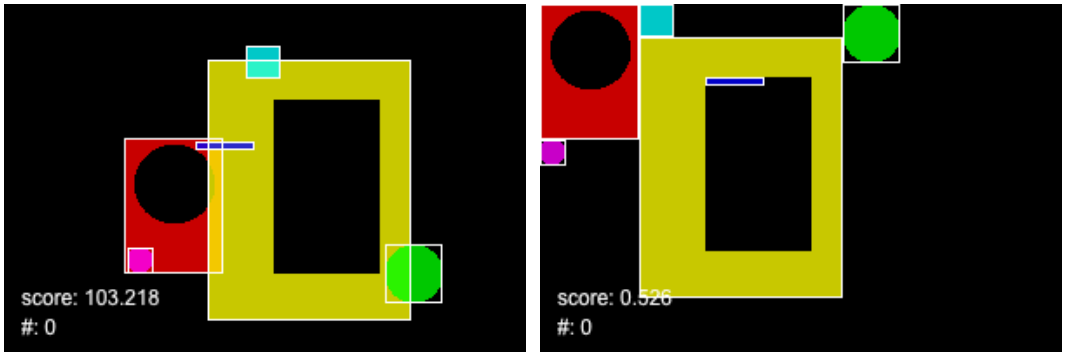


Figure 8: Left: original gene, right: gene after smart mutation.

3.8 Computing the distance between two genes

A useful feature to ensure that some diversity is kept in the population is to have a measurement of the distance d between two individuals. In our case, two factors come into play to define how similar two genes are:

- The sum of the euclidean distance d_e between the centers of the pieces in the first and second gene (in pixels).
- Wether the orientation of the pieces is the same in both genes. We define a variable r which is incremented by one every time a piece has different orientations in the two genes.

In order to convert these two parameters into a single scalar value, we once more use a weighted sum:

$$d = w_{d_e} \cdot d_e + w_r \cdot r \tag{3}$$

3.9 Selection of individuals for new generation

After a new individual is created using crossover and mutation, we have to decide wether it is added to the population or discarded. In our case, the population is divided into two distinct groups:

- The **elite** individuals: A small subset of the population which has the best scores ever computed. An individual is added to the elite if its score is better than one of the current elite. The worst elite is then discarded.
- The **diverse** individuals: Individuals that are responsible for maintaining a high variety in the population. An individual is added to the diverse population if its minimal distance to the elite individuals is larger than that of one of the previous diverse individuals. The diverse individual that is the most similar to the elite is then discarded.

Each time a new gene is added to the elite, the minimal distances of the diverse individuals to the elite are updated.

3.10 End condition for the GA

A fixed number of generations is used as a variable of the GA. The algorithm is stopped once the desired number of iterations is reached. This is not optimal if the algorithm converges rapidly, since multiple generations would be computed without much (or any) improvement.

4 Implementation

4.1 Choice of technologies

All of the implementation was done using Node.js[1], which allows to run JavaScript code from the command line. What is more, the TypeScript syntax was used in order to ensure proper typing throughout the project and reduce the probability of type-related bugs. Working with JavaScript is not the most efficient from a performance point of view, indeed a lower level, compiled language like C would be much faster. However, that language was mostly chosen in order to use the `image-js` open-source image analysis library [12], to which we widely contributed in the past. This ensures that we already have a good knowledge of image analysis and manipulation in the chosen language. What is more, this project acts as a complex test of the library and allows to identify some missing features or bugs. GitHub was used throughout the whole project as a version control and project management tool. All of the code is published publicly under MIT license in the following repository: https://github.com/opatiny/fawami.

4.2 Dataset

The dataset to test our algorithm is constituted of two types of images:

- Small test images created using Inkscape[2]: Small size images containing basic shapes such as rectangles and ellipses (see Figure 9). The goal of these images is to create tests with a low number of pixels, which allows to debug and test edge cases (such as pieces with holes).
- Actual sewing patterns exported as SVG images from the open-source parametric pattern design tool FreeSewing[3] (see Figure 24). These patterns contain a lot of annotations such as text and arrows (which have to be removed), but provide data relevant for the textile industry specifically.

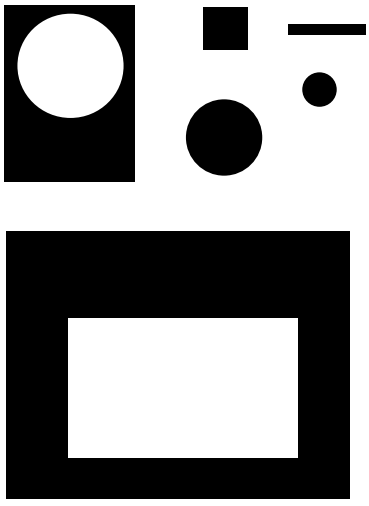


Figure 9: Example of one of the test images created using Inkscape (dimensions: 210x297mm). Some of the pieces contain holes.

[1] https://nodejs.org/en
[2] Inkscape is an open-source vector graphics design software: https://inkscape.org/
[3] https://freesewing.eu/

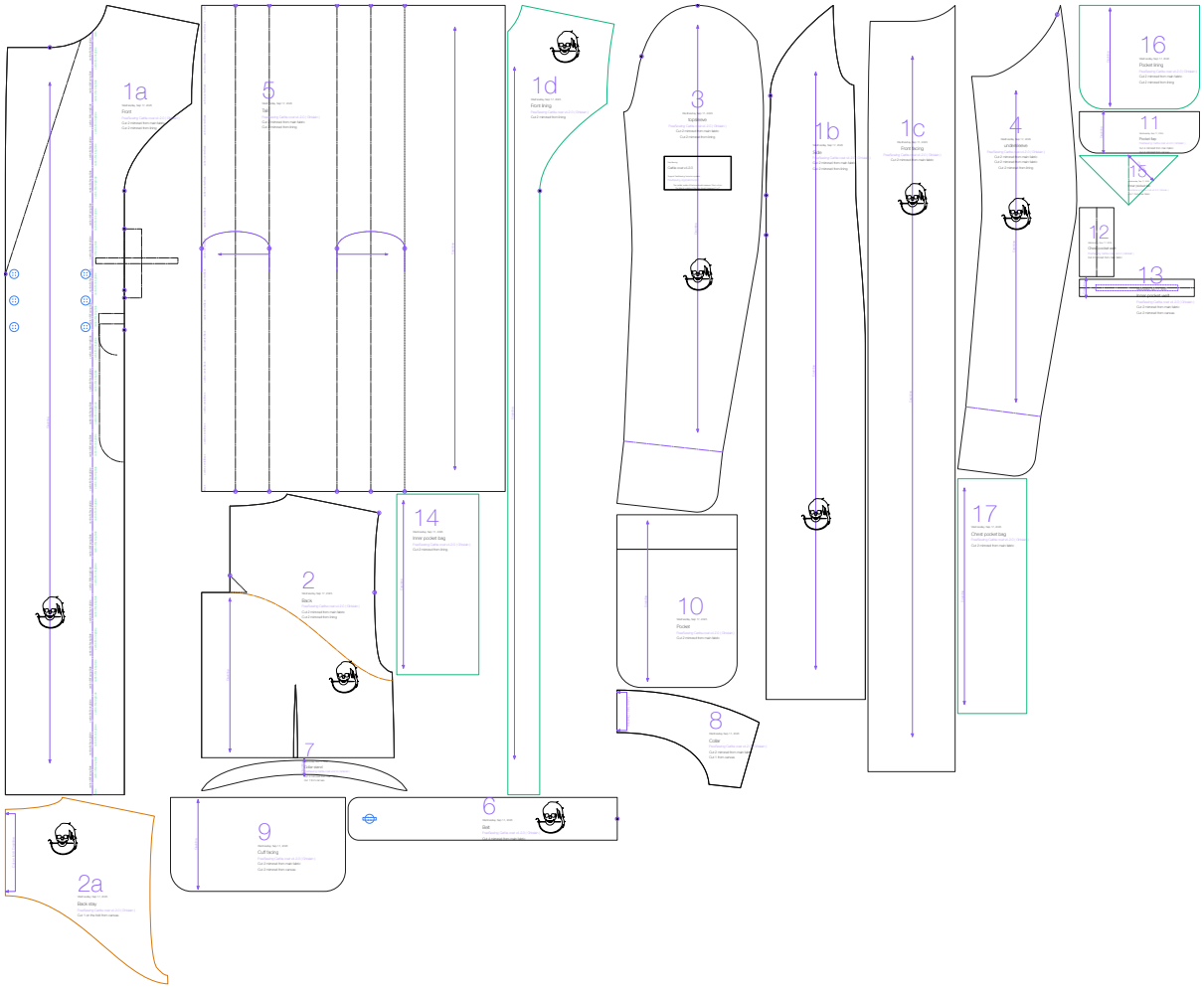


Figure 10: Example of the pattern "Carlita", exported from FreeSewing (dimensions: 1800x1480mm).

4.3 Convert SVG into a discrete image with desired resolution

The user must provide an SVG (Scalable Vector Graphics) file containing all of the pattern pieces, as well as the desired output resolution in pixels per cm (default: 10). The image must contain all of the pattern pieces as separate paths (which are not touching). The image can contain text and other annotations: they will be removed. What is more, the pieces can contain holes, which should be closed shaped, filled in white. The different shapes are identified and filled in black. They are then converted into a PNG image, which can finally be used to create an ‘image-js‘ image instance.



Figure 11: Left: input image in SVG format. Right: the discretized image with a given resolution in pixels per cm (in that case: $10\,\mathrm{px/cm}$). All annotations were removed, and the pieces were filled in black to facilitate the detection of the regions of interest (described in subsection 4.4).

4.4 Identify pattern pieces

Once the input pattern has been discretized, the different pattern pieces have to be separated. In order to do that, the `image-js` image's color model is set to gray (only one channel). Then, the gray image is transformed into a binary image (also called a **mask**): pixels above a given threshold have a value of one, otherwise they are set to zero. This allows to detect the areas that are within a pattern piece. The next step is to identify the regions of interest (ROIs) in the image. Each ROI has a `mask` property, which is a black and white image associated with the given pattern piece, as well as multiple metadata information. The ROIs are then used to generate an array of `PatternPiece` instances (see subsection 4.6). These objects can be drawn onto a fabric image, hence allowing to see what the current individual looks like. An example of this is shown in Figure 12.

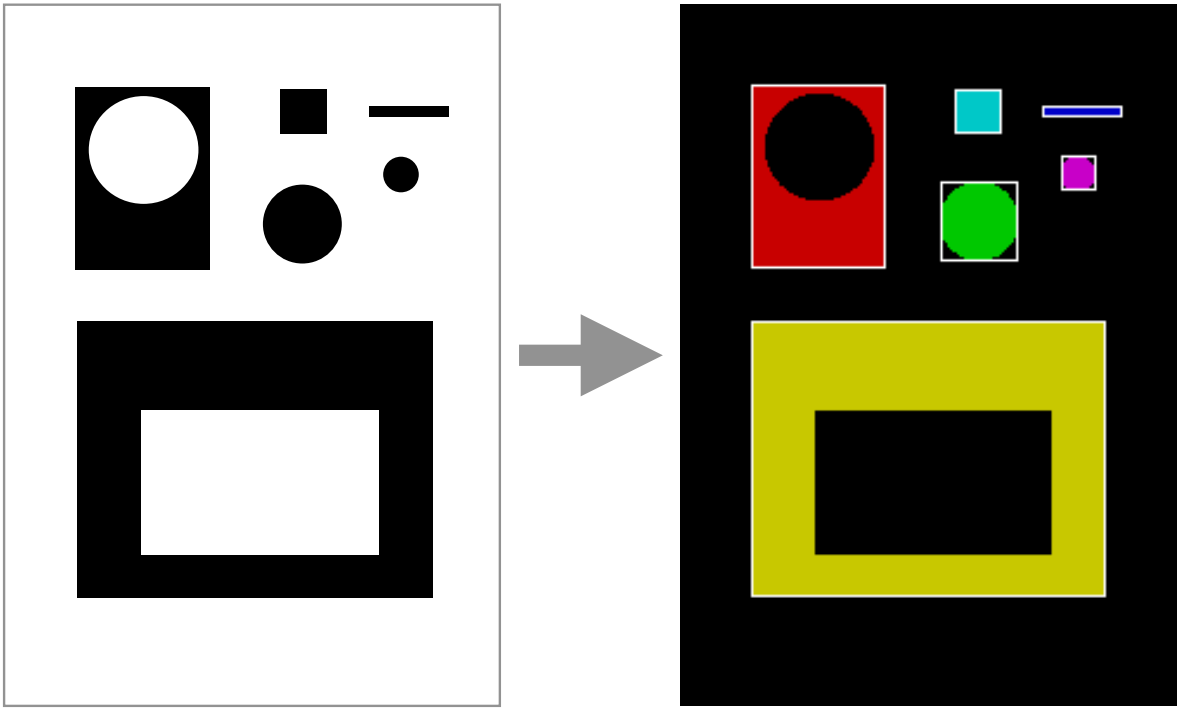


Figure 12: Left: input image in SVG format. Right: the extracted pattern pieces drawn on a fabric image, with their bounding rectangles in white. Each piece detected is drawn in a distinct color.

4.5 Overlap computation

The overlap computation is the most resource intensive part of the whole algorithm. Indeed, it requires to count the numbers of overlapping pixels of all the pieces every time that a gene is modified and its fitness must be recomputed. In order to optimize the computation, the following approach was used:

- Start with a counter of 0.
- First compute the positions of the bounding rectangles of the two pieces.
- If the BRs do not intersect, return 0.
- Else, compute the bounding rectangle of the intersection.
- For each pixel of the intersection rectangle, verify if it is within piece 1.
- If it is not in piece 1, continue (this step allows to increase the speed of the genetic algorithm significantly: around 10 %).
- Else, verify if it is in piece 2 as well.
- If the pixel is within both pieces, increase counter by 1.
- Once all pixels are processed, return the counter.

An example of the algorithm applied to simple pieces is shown in Figure 13. This method for overlap calculation allows to easily handle pieces with multiple holes. Moreover, the algorithm works for pieces with any cardinal orientation, because it uses utilities implemented on the `PatternPiece` class which return the value of a desired pixel in the rotated piece mask. However, extending this approach to more orientations would be difficult because of the aliasing effect which appears when rotating pixellized shapes.

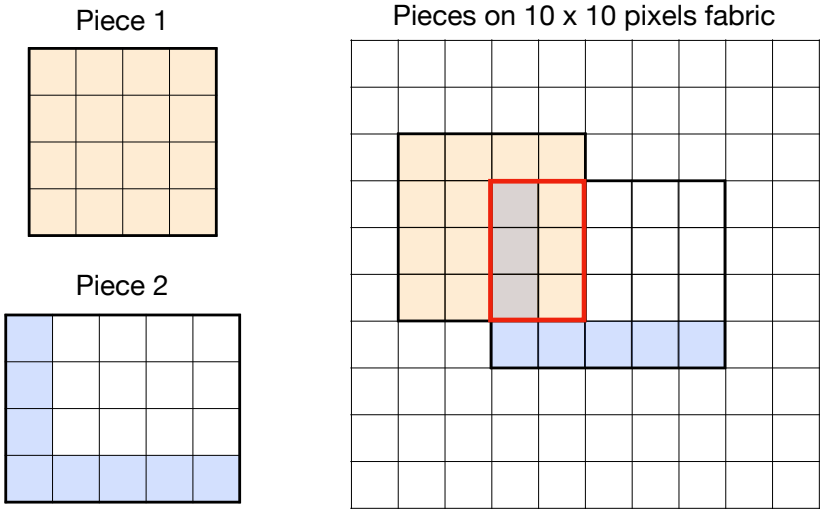


Figure 13: Computing the overlap of two small pieces placed on a fabric image. The intersection of the bounding rectangles is represented in red. The algorithm has to treat each of the 6 pixels within the intersection, and count the number of pixels in both pieces, in this case 3.

4.6 The `PatternPiece` class

We created a `PatternPiece` class[1] which facilitates the manipulation of the pieces to cut. The class has three main properties:

- `mask`: The binary mask corresponding to the pattern piece. This mask is unique, and it will be the same object for all pattern pieces that are created based on an original pattern piece (the pointer is copied instead of the data). This prevents from copying multiple times the same data, and reduces memory usage.
- `centerOrigin`: Origin of the piece's center pixel relative to the top-left corner of the fabric image.
- `orientation`: Orientation of the pattern piece.

Methods were implemented on the class in order to retrieve a given pixel from the rotated piece without having to actually create the rotated mask. However this only works for cardinal orientations.

[1] https://github.com/opatiny/fawami/blob/main/src/patternPiece/PatternPiece.ts

4.7 The `Gene` class

The `Gene` class[1] allows to represent one solution of the genetic algorithm. Each gene instance is associated with a fabric image, which must have the same resolution as the pattern pieces. The gene contains an array of pattern pieces, which all have their position and orientation properties set to indicate where they are placed on the fabric. What is more, the class has additional properties allowing to compute the gene's fitness using the `getFitness()` method. The fitness weights to use to compute the fitness should be provided when a new gene instance is created. Moreover, the gene has an `overlapMatrix` property which allows to store already computed values of pieces overlap. In case the gene is modified (some pieces are rotated or translated), the entries of the matrix corresponding to the modified pieces are set to -1, indicating that they have to be updated. In this way, we avoid recomputing the whole overlap matrix when only one piece is modified, such as in the `smartMutate` algorithm (see subsection 3.7).

4.8 The `GeneticAlgorithm` class

In order to have a better understanding of how a genetic algorithm works, a genetic algorithm class[2] was created from scratch. The goal was to make this class as generic as possible, and fully independent from the problem at hand, in order to be able to reuse it to solve other tasks. To create an instance of the class, the user must provide:

- An initial population
- A fitness function
- A crossover function
- A mutation function

All other parameters are optional. The main options are summarized in Listing 1. In order to get repeatable results for testing, it is possible to create a random generator with a seed, and pass it to the genetic algorithm class. That random generator will then be used in all the functions called within it. Moreover, the class has a method called `getNextGeneration`, which computes the next iteration of the genetic algorithm. Every time this method is called, the best gene is stored in the `bestScoredIndividuals` property. The function to compute the distance between two individuals is made optional, but it is necessary to implement an appropriate distance measurement for the problem being treated (such as the algorithm presented in subsection 3.8).

```
1 export interface InternalOptionsGA<Type> {
2   getDistance: DistanceFunc<Type>;
3   enableCrossover: boolean;
4   enableMutation: boolean;
5   initialPopulationSize: number;
6   populationSize: number;
7   eliteSize: number;
8   randomGen: Random;
9 }
```

Listing 1: Main properties of the `InternalOptionsGA` interface

[1] https://github.com/opatiny/fawami/blob/main/src/textileGA/Gene.ts
[2] https://github.com/opatiny/fawami/blob/main/src/gaLib/GeneticAlgorithm.ts

4.9 The `TextileGA` class

The `TextileGA` class is the highest level class implemented in this project. Internally, it will create an instance of the `GeneticAlgorithm` class, initialized correctly to deal with the pattern pieces placement problem. If not provided, an initial population of genes is automatically generated when a new instance of the class is created. The only mandatory variables for creating a new instance of the textile GA are a fabric image, and an array of pattern pieces. All of the hyperparameters of the genetic algorithm can be set when creating a new instance of the class. These parameters include the fitness and distance weights, the number of iterations of the GA, the mutation and crossover options, and more. All these parameters can be saved to a JSON file using the `saveConfig()` method. This feature is useful in order to store all the configuration relevant to a given optimization test. An additional option, `nbCuts`, was added, which specifies how many times the pattern pieces should be placed on the fabric. This feature allows to easily test the genetic algorithm with a larger number of pieces. Finally, the rotation of the pieces can be enabled with the `enableRotation` option.

5 Debug Strategies

As the project evolved, the amount of code increased proportionally. This made the detection of potential bugs more and more difficult. In order to facilitate the detection and prevention of bugs, multiple strategies were used.

5.1 Unit testing

Firstly, whenever possible, all functions are unit tested[1]. This ensures that each smaller bit of code indeed acts as expected, and allows to test edge cases as well. What is more, if the code has to be modified at some point, running all test cases and having them pass ensures that no previous functionality was broken by the new addition. The `image-js`[2] library also provides functionalities specifically for unit tests dealing with images. For instance, we can ensure that an image matches a previously generated snapshot.

5.2 Saving solution images and creating GIFs

In order to visualize the solutions, a function was implemented which allows to save an individual to an image. An RGBA image with the fabric dimensions is created, then all the pattern pieces are drawn on it with distinct colors. The pieces are semi-transparent, which allows to see the piece overlap more clearly. What is more, a whole population of individuals can be saved at once. Optionally, the images can be numbered, and the score of each individual can be displayed on the image, as can be seen in Figure 14. For some tests, the best gene images were saved and then converted into an animated GIF, which allowed to visualize the evolution of the algorithm.

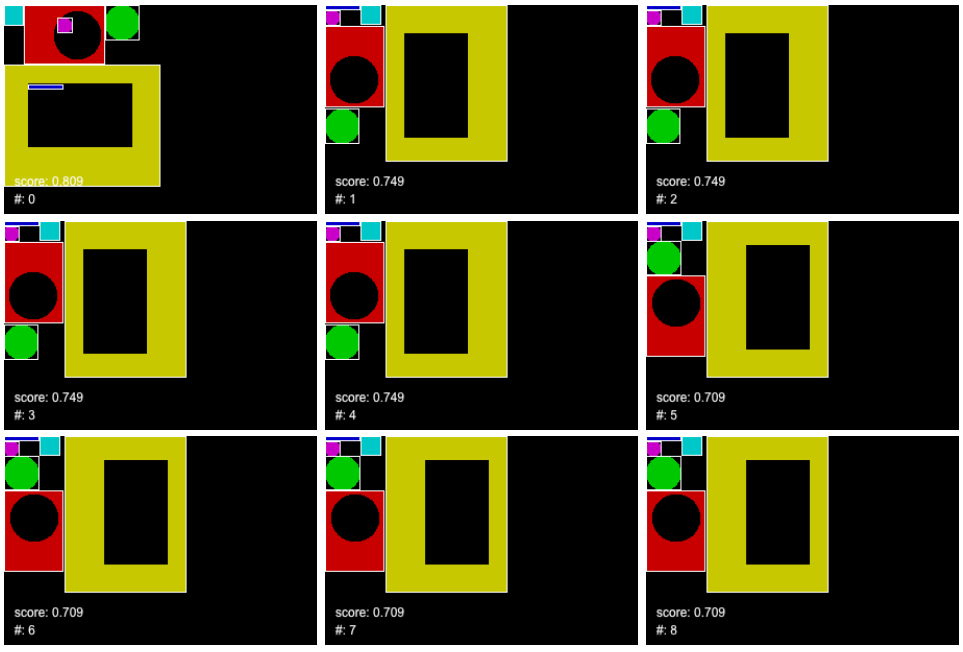


Figure 14: Saved images corresponding to the best solutions at each iteration of a basic optimization test.

[1] Using the Vitest testing framework: https://vitest.dev/
[2] Refer to subsection 4.1

5.3 Plotting convergence

A utility to plot the convergence of the GA was implemented as well. These plots are useful in order to adapt the number of iterations of the algorithm. As an example, the convergence plot corresponding to the best genes images shown in Figure 14 is shown in Figure 15.

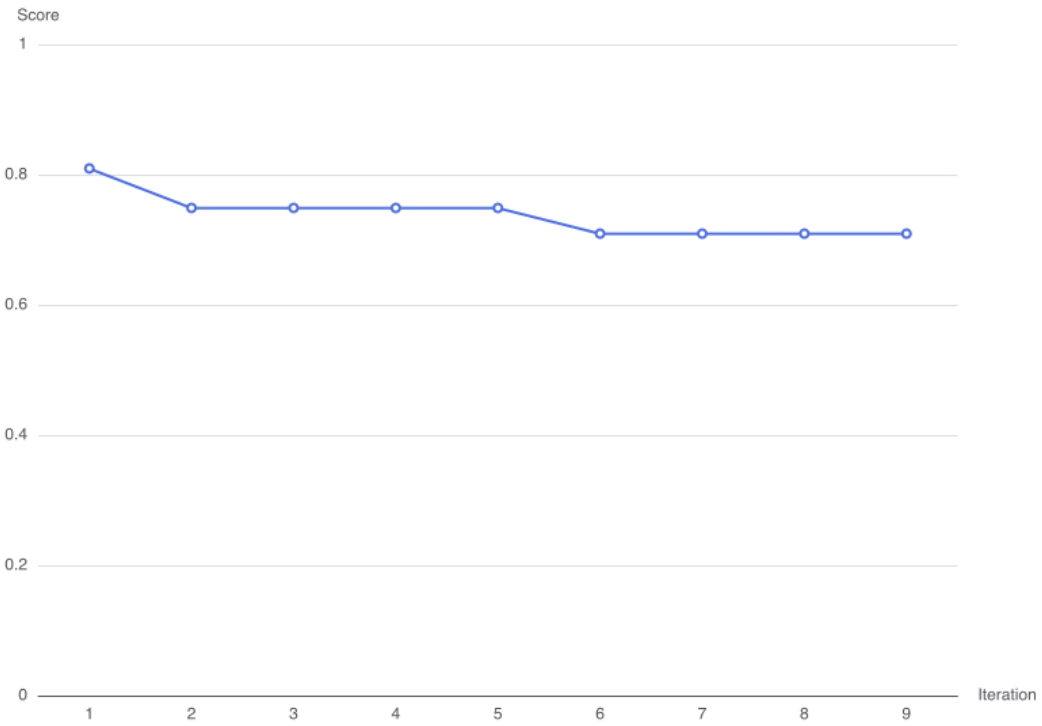


Figure 15: Convergence plot corresponding to the test shown in Figure 14

5.4 Distance heatmaps

One final graphical debug feature that was implemented is the possibility to save a distance heatmap of the individuals of a population. Imagine a population of ten individuals, then the heatmap is a grid of 10 x 10 cells, where each cell's color encodes the distance between two solutions. It is the graphical representation of the distance matrix of the population. Using these distance heatmaps allows us to inspect the diversity of the population at each iteration of the genetic algorithm. The heatmap of ten random individuals is illustrated in Figure 16.

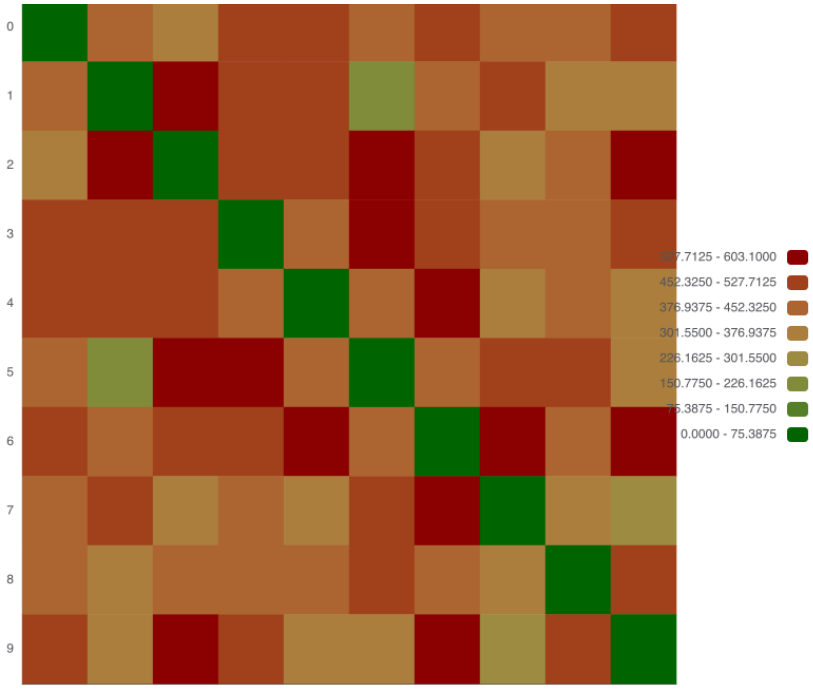


Figure 16: Distance heatmap of a random population of ten individuals. The legend is the distance between the two individuals.

5.5 Saving all relevant data to a test

An array of tests was run in order to analyze the behavior of the genetic algorithm. For each test, the following data was saved:

- All the parameters of the algorithm, using the `saveConfig()` function presented in subsection 4.9
- The images of the initial population, and optionally all the iterations
- The convergence plot of the genetic algorithm
- The distance heatmaps at each generation

Therefore, it is possible to come back to previously executed tests and understand the context in which they were run, making it easier to draw general observations on the behavior of the algorithm.

6 Results

6.1 Impact of the fitness weights

In this section, the GA is run with different values of the fitness function weights in order to identify the effect of each of the weights on the global results of the algorithm. The other options are kept constant for each test, and are summarized in Table 2. The test pattern used is a set of 6 geometric shapes, some of them containing holes (see Figure 17). The fabric on which to place the pieces has dimensions 30 x 20 cm. As a reminder, they are five different weights in the fitness function which can be modified, which correspond to the following variables: overlap area, used length, average row, average column and packing. In each of the subsequent tests, all variables which are not specified are set to zero.

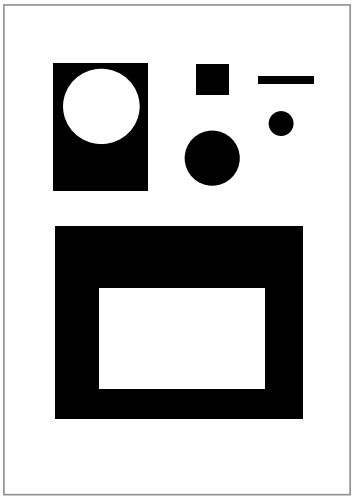


Figure 17: Input pattern for the fitness weights tests

Option	Value
Seed	0
Resolution	10 px/cm
Number iterations	1
Number cuts	1
Pieces rotation	OFF
Population size	10
Mutation translation amplitude	2 px

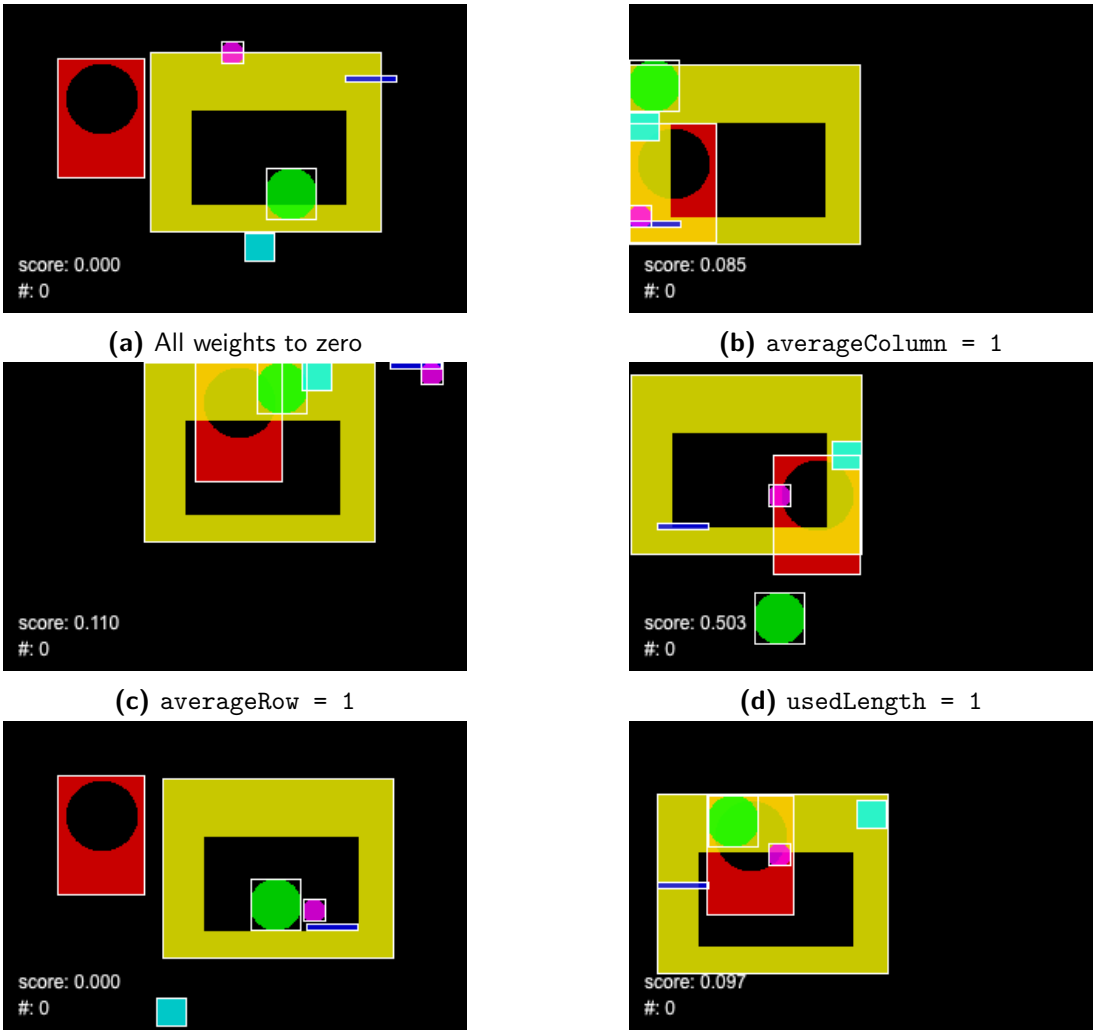
Table 2: Main GA options used for all the fitness weights tests

6.1.1 Testing each weight separately

In this section, each of the fitness weights is tested independently. The results demonstrate that they were implemented correctly (see Figure 18), since the behavior of the algorithm matches what is expected:

– The average row and average column weights push all the pieces respectively toward the top and the left of the fabric.

– The used length only acts on the right-most piece, and is limited by the widest piece (in this case the yellow rectangle).
– The overlap results in a solution without overlapping pixels.
– The packing compacts all the pieces. When no overlap weight is used, the pieces are all placed within the largest one.

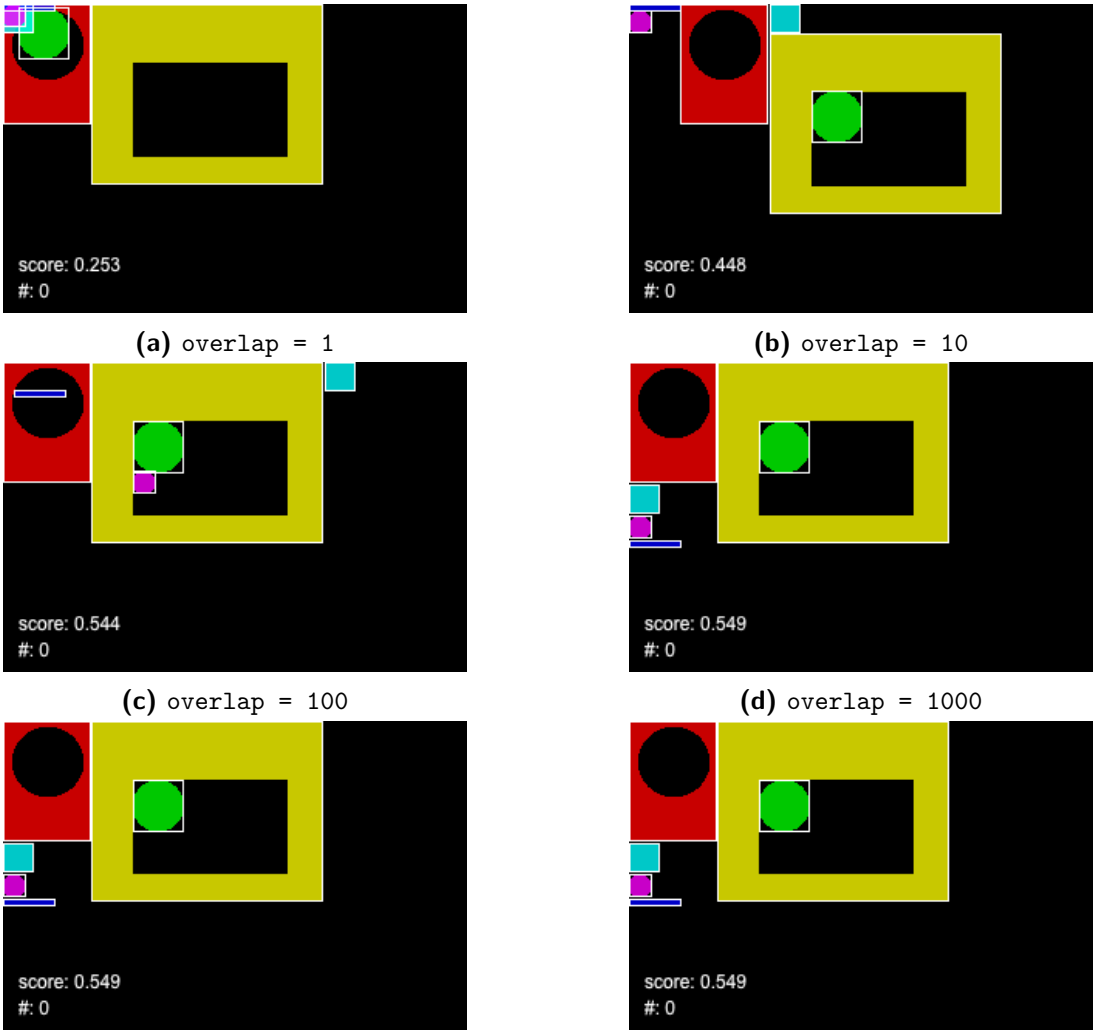


(a) All weights to zero
(b) `averageColumn = 1`
(c) `averageRow = 1`
(d) `usedLength = 1`
(e) `overlap = 1`
(f) `packing = 1`

Figure 18: Result of the GA when testing each fitness weight independently

6.1.2 Increasing overlap weight values

For this test, we combine three parameters: average row (set to 1), average column (set to 1) and the overlap. Various values of the overlap weight are tested. The results are shown in Figure 19. In this figure, one can see that, in that specific case, an overlap weight of 10 is sufficient to prevent overlap in the final solution. However, increasing the weight even more changes the final solution. Over a certain value, however, the solution remains the same: we observe a saturation phenomenon. Therefore, there seems to be an optimal value of the weights, which depends on the dimensions of the fabric and the pieces. Finding these optimal values consists in hyperparameter optimization, as presented in subsection 3.1.



(a) `overlap = 1` **(b)** `overlap = 10`

(c) `overlap = 100` **(d)** `overlap = 1000`

(e) `overlap = 1e4` **(f)** `overlap = 1e5`

Figure 19: Result of the GA with `averageRow = 1` and `averageColumn = 1`, and increasing overlap weights

6.1.3 Adding packing

In section 6.1.2, we saw that the overlap weight has to be significantly larger than the row and column weights in order to prevent overlap in the mutated individuals. They are however other cases where adding a non-zero weight for the packing allows to improve the solution even more, as shown in Figure 20. In order to demonstrate that, some parameters had to be modified: rotation enabled, number cuts: 3, and number iterations: 10. The weights used are summarized in Table 3.

Weight	Value
Average row	1
Average column	1
Used length	0
Overlap	1e5
Packing	Varied

Table 3: Fitness weights for the increasing packing weight tests



(a) `packing = 0` **(b)** `packing = 1`

Figure 20: Result of the GA without and with packing. Having a non-zero packing weight results in a more compact solution (packing of 59 % vs 52 %).

6.1.4 Best fitness weights

Using a trial and error method, we consider the fitness weights presented in Table 4 as good hyperparameter values for the tests we realized.

Weight	Value
Average row	1
Average column	1
Used length	0
Overlap	1000
Packing	1

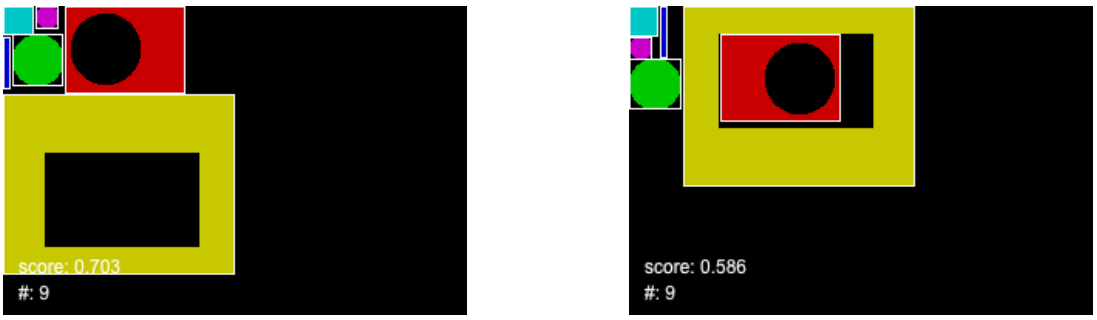
Table 4: Optimal fitness weights found with trial and error method

6.2 Varying the seed

In this test, we want to demonstrate the impact of the random generator's seed value on the results of the algorithm. The optimal fitness weights from subsubsection 6.1.4 are used, and the genetic algorithm's parameters are summarized in Table 5. The final results may vary on the seed, as shown in Figure 21. Indeed, the genetic algorithm returns a local minimum in the solution space, and the minimum toward which the algorithm converges depends the values of the random generator.

Option	Value
Seed	Varied
Resolution	10 px/cm
Number iterations	10
Number cuts	1
Pieces rotation	ON
Population size	10
Mutation translation amplitude	2 px

Table 5: Main GA options used for all the fitness weights tests



(a) `seed = 0` **(b)** `seed = 1`

Figure 21: Result of the GA when varying the random generator's seed

6.3 Testing various patterns with the same parameters

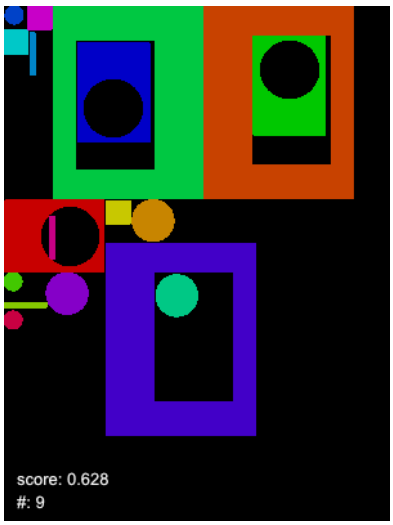
One common pitfall with machine learning is overfitting: optimizing the parameters of an algorithm to get good results for a specific training dataset, but leading to poor results whenever tested on new data. In this section, the exact same parameters were used with three different geometric shapes patterns (see Figure 23), in order to see how the algorithm deals with different types of shapes. The weights and options are summarized in Table 6 and Table 7. The results are visible in Figure 22. The following observation is drawn: the algorithm becomes slower as the fabric size and number of pieces increase.

Weight	Value
Average row	1
Average column	1
Used length	0
Overlap	1000
Packing	0.1

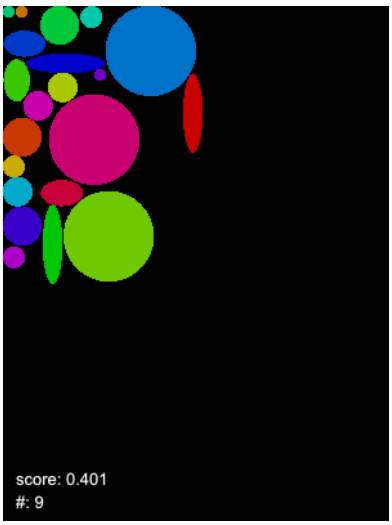
Table 6: Fitness weights used for the same parameters test

Option	Value
Seed	0
Resolution	10 px/cm
Number iterations	10
Number cuts	3
Pieces rotation	ON
Population size	10
Mutation translation amplitude	2 px

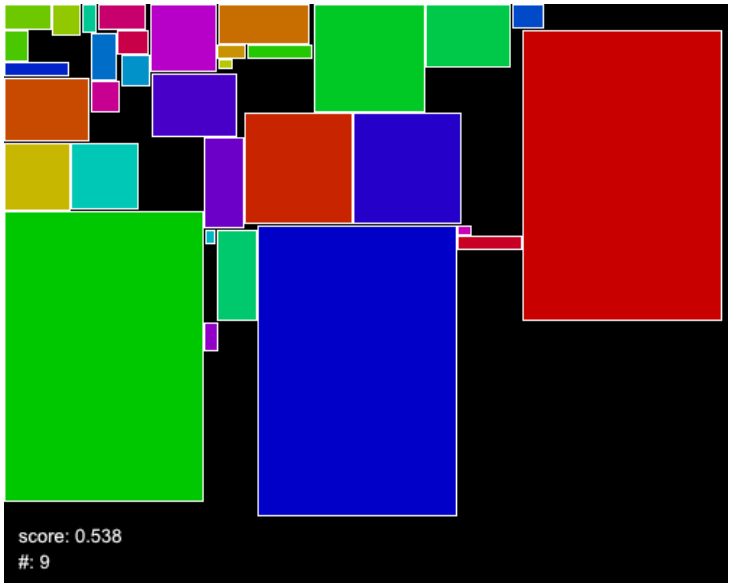
Table 7: Main GA options used for the same parameters test



(a) Pattern: holes (18 pieces), fabric: 30 x 40 cm, run time: 27.5 s, packing: 52 %

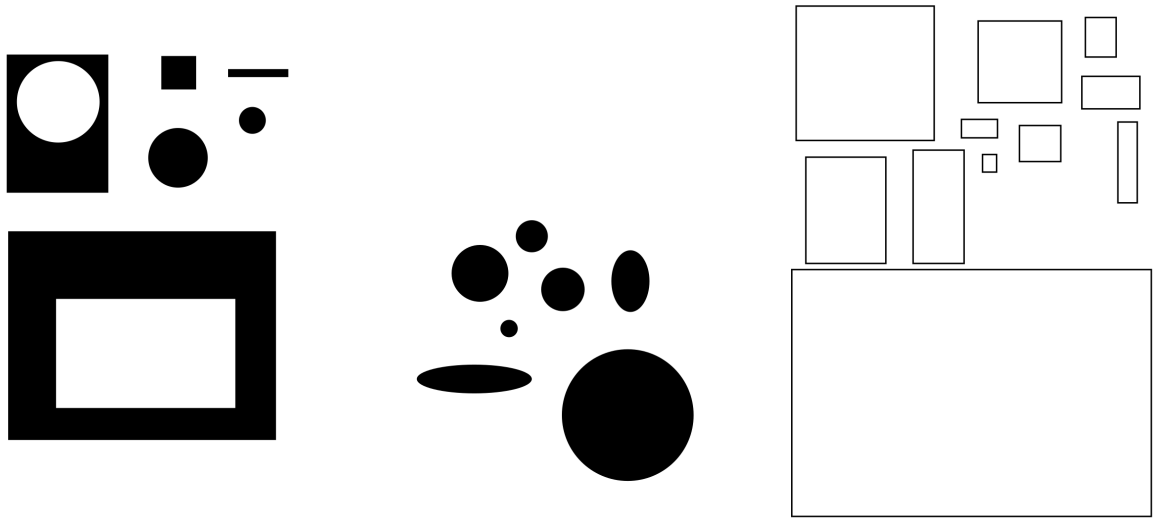


(b) Pattern: ellipses (21 pieces), fabric: 30 x 40 cm, run time: 11.2 s, packing: 59 %



(c) Pattern: rectangles (33 pieces), fabric: 50 x 40 cm, run time: 97.0 s, packing: 72 %

Figure 22: Results of the same parameters test, each pattern is cut three times



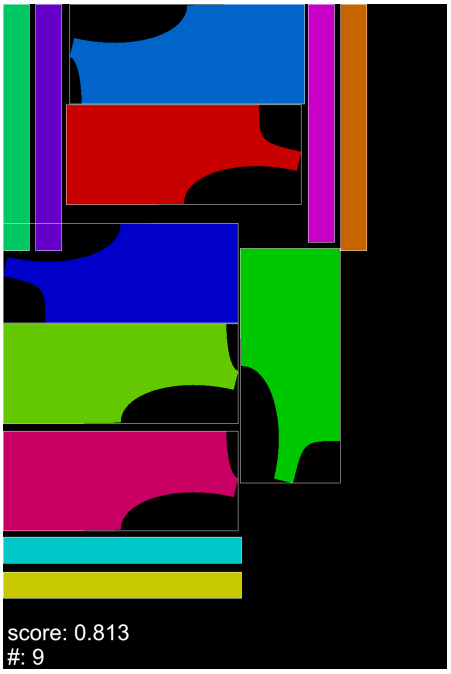
(a) Shapes with holes **(b)** Ellipses **(c)** Rectangles

Figure 23: The three patterns used in this test

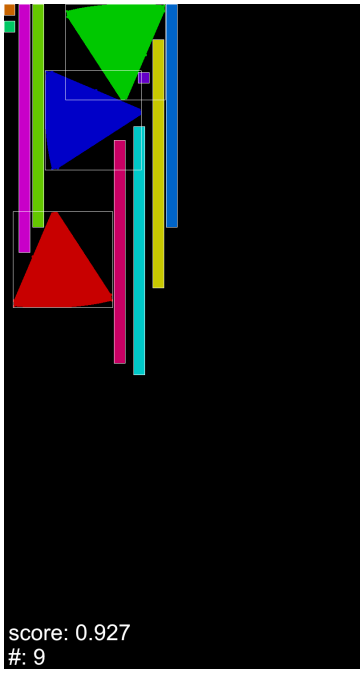
6.4 FreeSewing patterns

In this section, the algorithm is tested on some real sewing patterns, exported from the FreeSewing platform. The fitness weights are the optimal weights presented in subsubsection 6.1.4. The same options are used as in Table 7, except for the translation amplitude, which is set to 20[1]. We test three different patterns: Aaron (sleeveless top, 4 pieces), Bee (bikini top, 4 pieces) and Carlita (coat, 21 pieces). Because of the number of pieces of Carlita, that pattern is only cut once. The two other ones are cut three times. The results are shown in Figure 24.

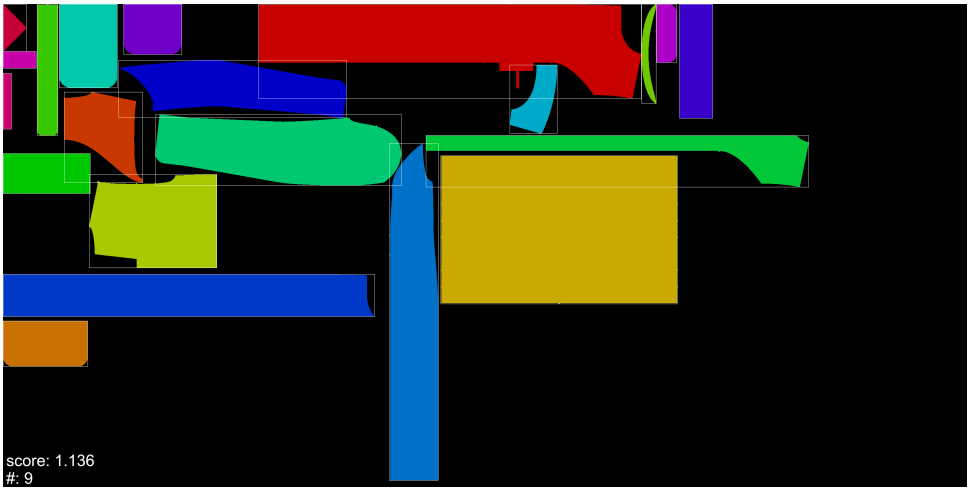
[1]With `translationAmplitude = 2` the algorithm takes more than 10 minutes to run with the Aaron pattern)



(a) Pattern: 3x Aaron, fabric: 150 x 100 cm, run time: 108 s, packing: 62 %



(b) Pattern: 3x Bee, fabric: 150 x 80 cm, run time: 25 s, packing: 51 %



(c) Pattern: Carlita, fabric: 300 x 150 cm, run time: 579 s, packing: 42 %

Figure 24: Testing the GA on FreeSewing patters

6.5 Local minimums

While testing the algorithm, we stumbled into the classical heuristics issue of local minima. In the scope of this project, the following cases were observed:

- When pieces have holes, two pieces can have overlap, but be stuck in a local minimum where whatever the movement direction is the overlap would increase (see Figure 25).
- Sometimes pieces get nicely packed, but then even if the whole set of pieces could still be moved top-left, the pieces cannot be moved one after the other without resulting in a worse score. Indeed, moving a piece would result in decreasing the packing. This issue might be a matter of the ratio between row and column weights, and packing weight. An example of this behavior can be seen in Figure 26.

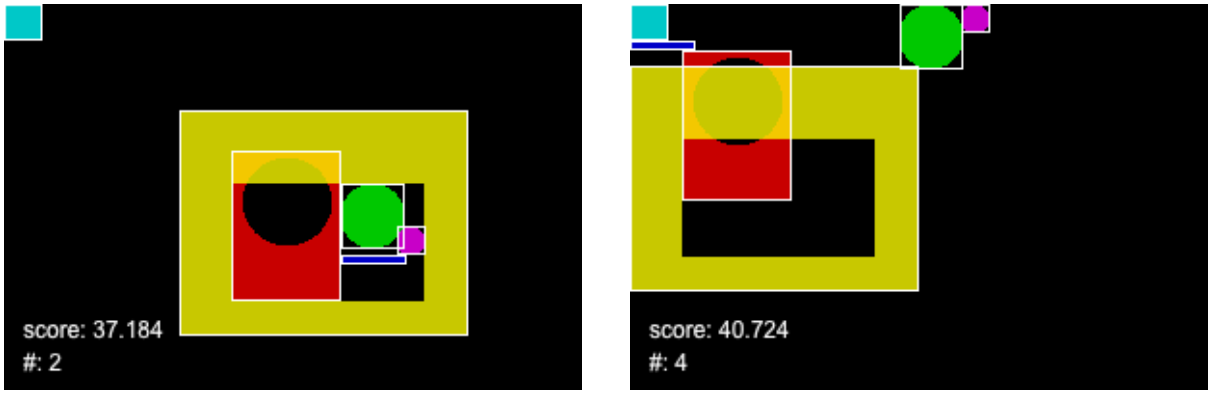


Figure 25: Issues with holes: result contains overlap but pieces cannot be moved without increasing the overlap even more

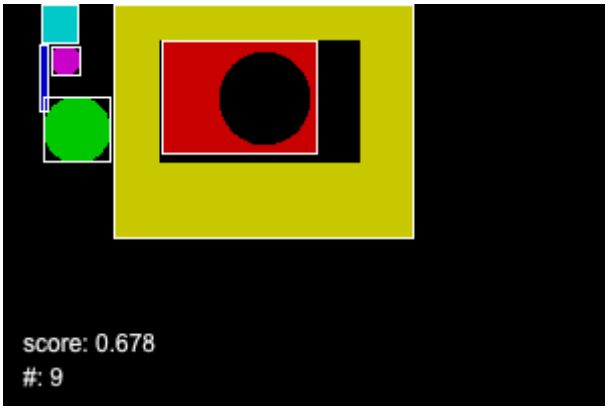


Figure 26: Local minimum caused by the packing weight being too large, which prevents the pieces to be moved to the left

7 Discussion

Firstly, utilities to extract pattern pieces from an SVG image were created, and allow to specify a desired piece resolution in pixels per centimeter. Moreover, the image processing tools are robust to text and other annotation on the pattern images, which they will automatically detect and remove. Then, a genetic algorithm for optimizing pattern pieces nesting on a rectangular piece of fabric was successfully implemented. The algorithm handles pieces cardinal rotations, as well as recursive nesting of pieces within each other. What is more, there is no limitation on the pattern piece shapes (such as being polygonal, or convex), since they are rasterized for the overlap computation. The fitness function being a weighted sum of various parameters leads to a lot of flexibility, but finding optimal weights is a challenge, with a risk of overfitting. Having the population split in two categories (elite and diverse individuals) ensures that a sufficient diversity is kept within the population. However, more tests could be done to establish the optimal number of individuals, as well as the ratio between elite and diverse genes. As it is, the algorithm is far from competing with existing solutions, which result in packing of around 80 %. Indeed, our average packing is of approx. 60 %. However, all of the keystones of the genetic algorithm were implemented, and could be modified at will in order to increase the performance (reduce runtime or enhance packing). What is more, the various graphic debug tools that were developed allow to inspect the results, and gain better understanding of the optimization process, which would be difficult otherwise. To conclude, all of the requirements for this project are fulfilled: we created a modular environment to test any fitness, crossover and mutation functions to solve the nesting problem.

8 Perspectives

This section gathers some possible improvement strategies for the genetic algorithm as it is currently implemented. Indeed, multiple aspects of the project could still be investigated further to improve the performances.

8.1 Final overlap is non zero

Throughout our tests, there were cases where the final solution still contained some pieces overlap. For instance, this was happened in Figure 25, despite the overlap weight being set to 1000. In order to remove any residual overlap, we could take the smaller of the pieces that still has an overlap and place it at the bottom-right corner of the fabric. After the solution goes through an additional smart mutation, there would be no residual overlap.

8.2 Handle increasing resolution (local search)

As pointed out in the state of the art, and later on experienced in subsection 6.4, the raster model shows some significant performance issues for larger number of pattern pieces. An approach to solve this would be to compute a first gross approximation using a lower resolution, and then increase the resolution for the final iterations, fine-tuning the position of the parts. Other hyperparameters, such as translation amplitude in the mutation, could also be adapted throughout the optimization.

8.3 Randomly pick pieces to crossover

Currently, one point crossover is used. Therefore, the pieces in the middle of the gene cannot be modified without changing the beginning or the end as well, which is sometimes counter productive. We propose two alternative crossover ideas which would prevent this issue:

1. Randomly pick the pieces to swap between the two genes. The number of pieces to swap could be a hyperparameter, or be chosen at random.
2. Randomly pick and index to start the crossover, and swap N pieces between the two parent genes.

8.4 Mutation: swap and rotate pieces

The current mutation algorithm only applies translation to the pieces. Therefore, the orientation of the pieces is fully defined by the orientations of the initial population. Depending that population, some of the pieces could not be placed in their optimal orientation[1]. What is more, our algorithm has the pitfall of getting stuck in local minimums. In order to fix both these issues, we propose to modify the mutation algorithm in order to add random variations of the pieces orientations, and to occasionally swap some of the pieces locations. Having larger changes in the genes should allow to explore a bigger fraction of the search space.

8.5 Utility to make an array of tests

The tests we realized so far only used a small number of images, which leads to qualitative results. It would be interesting to implement a tool which allows to input an array of pattern images, as well as the corresponding fabric dimensions, and which would run the GA on all of them and compute global statistics like average runtime and packing. For instance, an array of 100 FreeSewing patterns could be used, which would allow to optimize the hyperparameters specifically for this type of patterns.

[1] It is the case in Figure 24b, where the three triangles could fit together more compactly if their orientation was different.

9 Conclusions

In the scope of this project, a genetic algorithm is successfully implemented from scratch to solve the nesting problem, with a special focus on the textile industry fabric waste. Firstly, utilities to separate the pieces of an input SVG pattern are implemented. Then, the pieces as well as the fabric are rasterized into a discrete matrix with the desired resolution in px/cm. That data can subsequently be passed down to the genetic algorithm itself, which returns the optimal positions of the pieces on the fabric, as well as their cardinal orientations. Custom fitness, mutation and crossover functions are designed to solve the specific optimization problem. Pieces can contain holes, and can be nested one within the other. The algorithm is tested on sets of geometric shapes, and on actual sewing patterns. The quality of the results varies largely from one test to another. Final packing ranges from 40 to a maximum of 70 %. The run time of the algorithm increases with the number of pieces to place on the fabric, as well as fabric dimensions and resolution.

As a final note for this project, we would like to come back to the concept of pattern pieces layout optimization itself. Indeed, nesting algorithms will always result in some amount of waste that cannot be reduced, and that will be defined by the pattern design. This issue is well summarized in [2]:

> "Designers conceptualise the garment design, and pattern makers create the patterns to be cut from a fabric roll in the most efficient way possible, which is still inefficient. Therefore, the design mainly dictates the amount of waste generated during the production stage of the garment."

Therefore, in order to reduce textile waste even more, the way clothes are designed would have to be changed, creating pattern pieces that fit together without (or with hardly any) gaps: this is the essence of zero waste fashion design. The patterning and the layout optimization would become interlinked processes, allowing to get the best from the two waste minimization approaches [2].

References

[1] Nancy Cardona. *Fast Fashion Statistics 2025*. July 24, 2024. URL: https://www.uniformmarket.com/statistics/fast-fashion-statistics (visited on 12/30/2025).

[2] Nesma ElShishtawy, Pammi Sinha, and Julia A. Bennell. "A comparative review of zero-waste fashion design thinking and operational research on cutting and packing optimisation". In: *International Journal of Fashion Design, Technology and Education* 15.2 (May 4, 2022). Publisher: Taylor & Francis, pp. 187–199. ISSN: 1754-3266. DOI: 10.1080/17543266.2021.1990416. URL: https://doi.org/10.1080/17543266.2021.1990416 (visited on 11/20/2025).

[3] Holly McQuillan. "Zero Waste Design Thinking". PhD thesis. May 20, 2019.

[4] Bonfim A. Junior, Plácido R. Pinheiro, and Rommel D. Saraiva. "A Hybrid Methodology for Nesting Irregular Shapes: Case Study on a Textile Industry". In: *IFAC Proceedings Volumes*. 6th IFAC Conference on Management and Control of Production and Logistics 46.24 (Sept. 1, 2013), pp. 15–20. ISSN: 1474-6670. DOI: 10.3182/20130911-3-BR-3021.00056. URL: https://www.sciencedirect.com/science/article/pii/S1474667016321590 (visited on 11/20/2025).

[5] Baosu Guo et al. "Two-dimensional irregular packing problems: A review". In: *Frontiers in Mechanical Engineering* 8 (Aug. 10, 2022). Publisher: Frontiers. ISSN: 2297-3079. DOI: 10.3389/fmech.2022.966691. URL: https://www.frontiersin.org/journals/mechanical-engineering/articles/10.3389/fmech.2022.966691/full (visited on 11/20/2025).

[6] Benny Nielsen and Allan Odgaard. "Fast Neighborhood Search for the Nesting Problem". PhD thesis. Denmark: University of Copenhagen, Feb. 14, 2003. 109 pp. URL: https://di.ku.dk/forskning/Publikationer/tekniske_rapporter/tekniske-rapporter-2003/03-03.pdf (visited on 11/20/2025).

[7] Jiří Kubalík, Jiří Lažanský, and Petr Zikl. "Layout Problem Optimization Using Genetic Algorithms". In: *Knowledge and Technology Integration in Production and Services: Balancing Knowledge and Technology in Product and Service Life Cycle*. Ed. by Vladimír Mařík, Luis M. Camarinha-Matos, and Hamideh Afsarmanesh. Boston, MA: Springer US, 2002, pp. 493–500. ISBN: 978-0-387-35613-6. DOI: 10.1007/978-0-387-35613-6_56. URL: https://doi.org/10.1007/978-0-387-35613-6_56 (visited on 04/17/2025).

[8] Hanan Lutfiyya et al. "Composite stock cutting through simulated annealing". In: *Mathematical and Computer Modelling* 16.1 (Jan. 1, 1992), pp. 57–74. ISSN: 0895-7177. DOI: 10.1016/0895-7177(92)90078-Y. URL: https://www.sciencedirect.com/science/article/pii/089571779290078Y (visited on 01/08/2026).

[9] S. Vorasitchai and S. Madarasmi. "Improvements on layout of garment patterns for efficient fabric consumption". In: *Proceedings of the 2003 International Symposium on Circuits and Systems*. ISCAS 2003. Vol. 4. Bangkok, Thailand: DBLP, June 25, 2003, pp. 552–555. ISBN: 978-0-7803-7761-5. DOI: 10.1109/ISCAS.2003.1206139.

[10] Stefan Jakobs. "On genetic algorithms for the packing of polygons". In: *European Journal of Operational Research* 88.1 (Jan. 6, 1996), pp. 165–181. ISSN: 0377-2217. DOI: 10.1016/0377-2217(94)00166-9. URL: https://www.sciencedirect.com/science/article/pii/0377221794001669 (visited on 11/20/2025).

[11] Hussain Alibrahim and Simone A. Ludwig. "Hyperparameter Optimization: Comparing Genetic Algorithm against Grid Search and Bayesian Optimization". In: *2021 IEEE Congress on Evolutionary Computation (CEC)*. 2021 IEEE Congress on Evolutionary Computation (CEC). Kraków, Poland: IEEE, June 28, 2021, pp. 1551–1559. ISBN: 978-1-7281-8393-0. DOI: 10.1109/CEC45853.2021.9504761. URL: https://ieeexplore.ieee.org/document/9504761/ (visited on 12/10/2025).

[12] Michael Zasso. *image-js/image-js: Image processing and manipulation in JavaScript*. Version v1.3.0. Dec. 2, 2025. URL: https://github.com/image-js/image-js (visited on 12/30/2025).